

# Introduction to Monte Carlo simulations

# Why Monte Carlo?

- Many-body systems have enormous configuration spaces (number of possible bitstrings)
- For a system of 10 particles, we need to consider all  $2^{10}$  configurations
- Exact calculations often impossible
- Monte Carlo uses random sampling to estimate averages

# Monte Carlo method

- Sample configurations according to a probability distribution
- Compute observables as averages over sampled states
- Goal: approximate ensemble averages

# Statistical Mechanics Connection

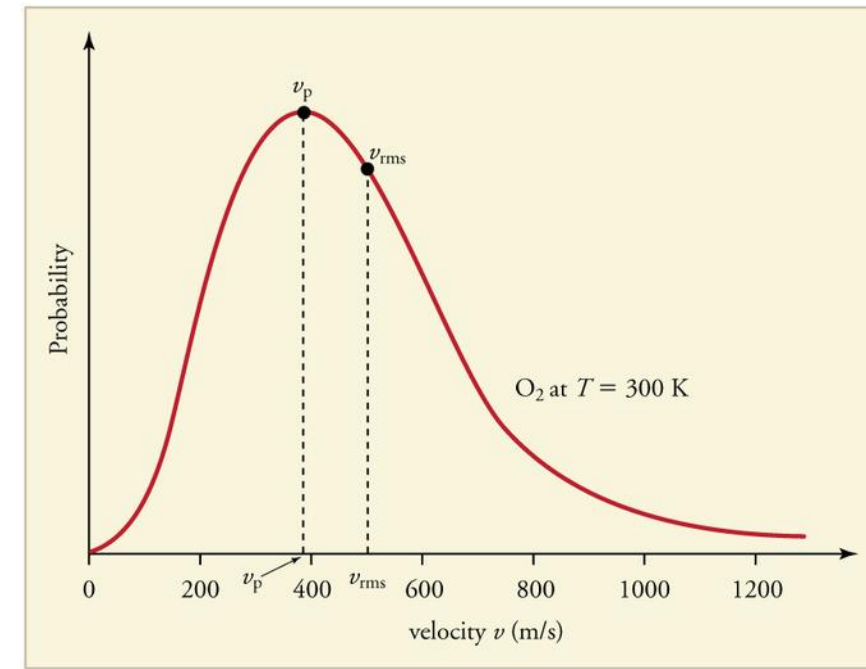
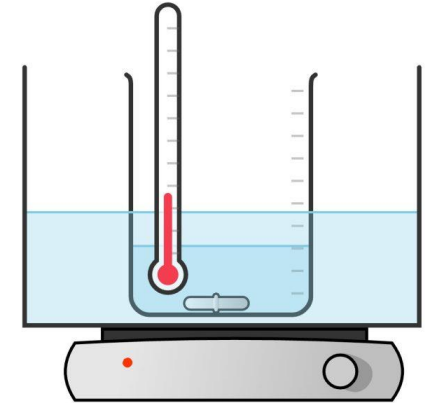
- System at temperature  $T$
- Probability of bitstring controlled by Boltzmann distribution

$$P(\alpha) = \frac{e^{-\beta E(\alpha)}}{Z} = \frac{e^{-\beta E(\alpha)}}{\sum_{\alpha'} e^{-\beta E(\alpha')}}$$

where  $\beta = 1/kT$ ,  $Z$  is the partition function

- Probability of bitstring  $\alpha$  depends on its energy

⇒ very high energy bitstrings (which have negligible probabilities due to the Boltzmann distribution) will not contribute to the overall sum

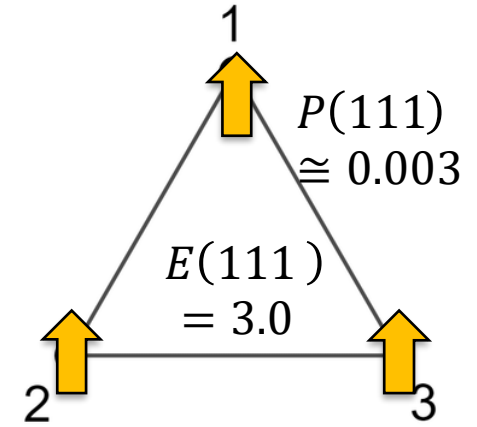
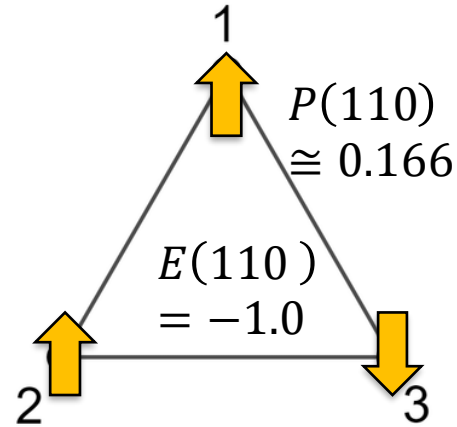
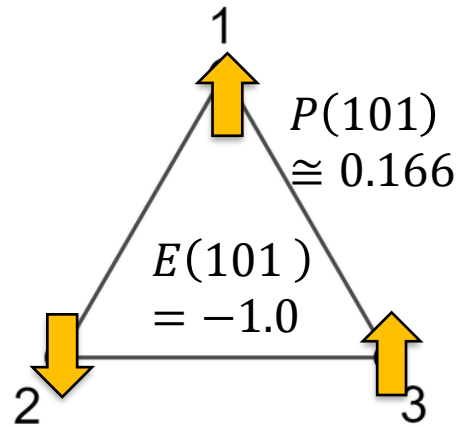
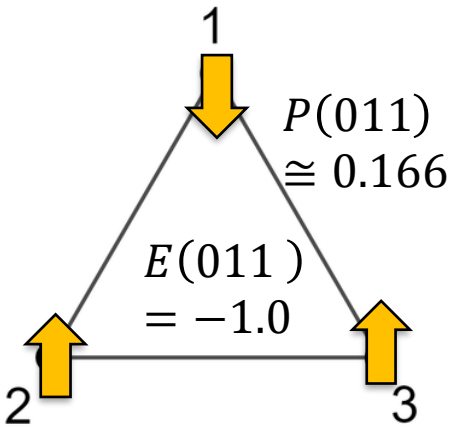
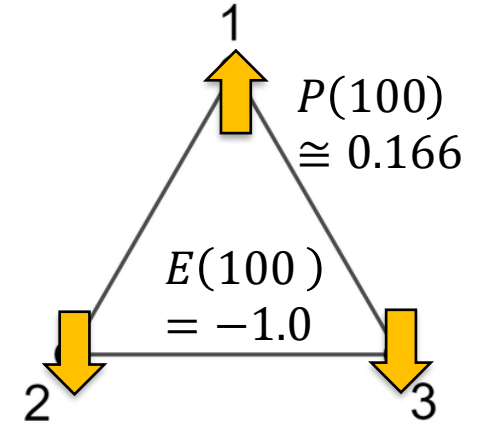
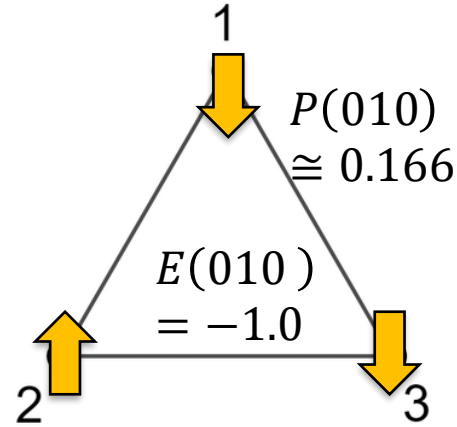
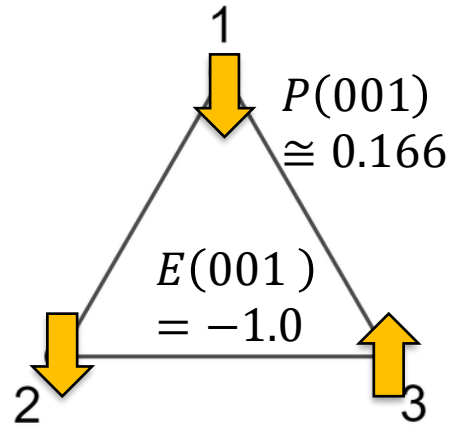
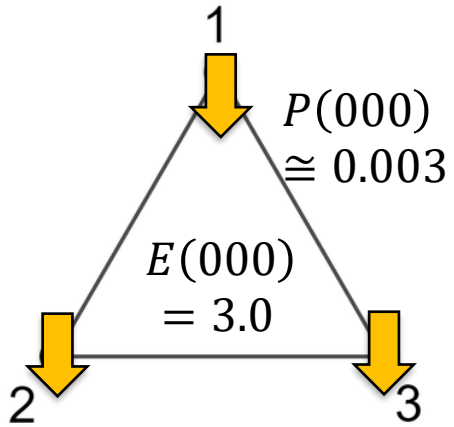


# Example

- $H = \sum_{(i,j) \in E} J_{ij} s_i s_j$ ; assume  $J_{ij} = 1.0$  for all  $i, j$
- $P(\alpha) = \frac{e^{-\beta E(\alpha)}}{Z} = \frac{e^{-\beta E(\alpha)}}{\sum_{\alpha'} e^{-\beta E(\alpha')}} \quad (Z: \text{partition function})$

Assume  $\beta = 1.0$

$$Z = e^{-3.0} + e^{+1.0} + e^{+1.0} + e^{+1.0} + e^{+1.0} + e^{+1.0} + e^{-3.0} \cong 16.4$$



# Metropolis Sampling

- Take these probabilities into account
- Generate bitstrings with probabilities according to the Boltzmann distribution

$$P(\alpha) = \frac{e^{-\beta E(\alpha)}}{Z}$$

- Earlier, when sampling *all* bitstrings, we computed the average value as:  
 $\langle E \rangle = \sum_{\alpha} P(\alpha) E(\alpha)$
- Here, if each bitstring already has the thermodynamic probability of being sampled, our average value would reduce to a simple average over the samples:

$$\langle E \rangle = \frac{1}{M} \sum_{\alpha \in \text{Trajectory}} E(\alpha)$$

- To carry out this special sampling, we construct a Markov chain or process

# Metropolis Algorithm Steps with Markov Chains

- *Walk* randomly through configuration (bitstring) space
- For each configuration that we visit, compute properties
- From our current configuration, propose a new randomly chosen configuration and decide whether or not to visit that new configuration

Algorithm:

1. Start from an initial configuration  $\alpha$
2. Propose a random move to a new configuration  $\beta$
3. Compute energy change  $\Delta E = E(\beta) - E(\alpha)$
4. Accept or reject using Metropolis rule

# Metropolis Acceptance Rule

- Propose a random move to a new configuration  $\beta$  from  $\alpha$
- 1. If  $\beta$  is lower in energy than  $\alpha$  ( $\Delta E \leq 0$ ), then we will visit the new configuration with 100% probability, i.e.,

$$W(\alpha \rightarrow \beta) = 1.0$$

- 2. Else, if  $\beta$  is *higher* in energy ( $\Delta E > 0$ ), visit the new configuration with probability  $e^{-\beta\Delta E}$ , i.e.,

$$W(\alpha \rightarrow \beta) = e^{-\beta(E(\beta)-E(\alpha))}$$

Why use this?

- Makes our simulations faster at the cost of statistical noise
- Markov chain converges to Boltzmann distribution

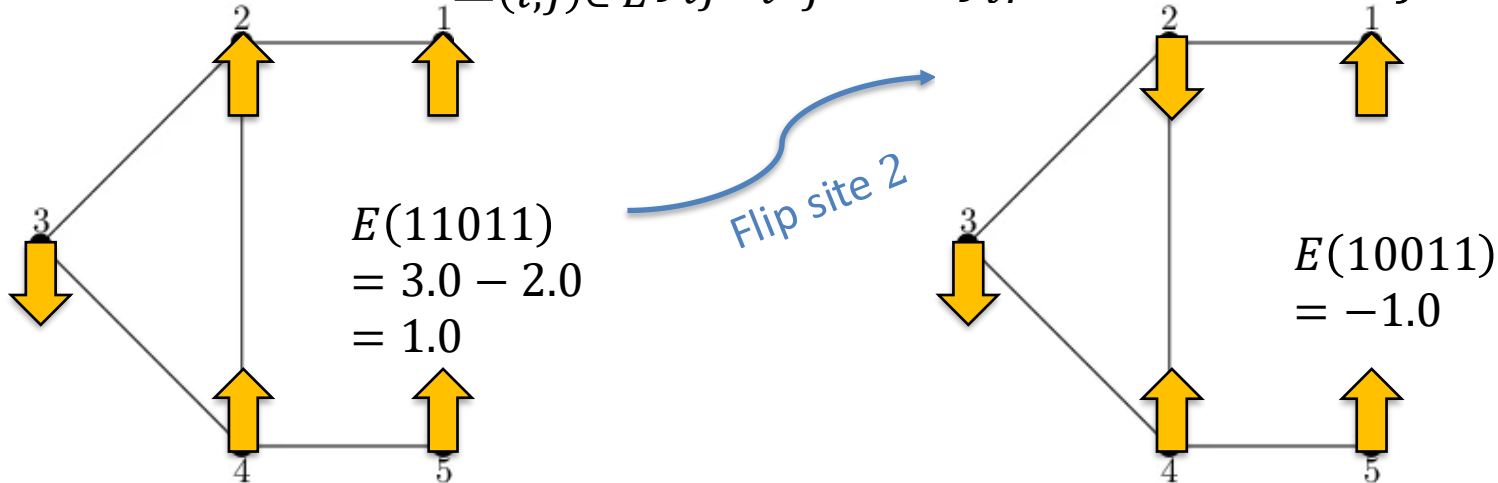


# Pseudocode

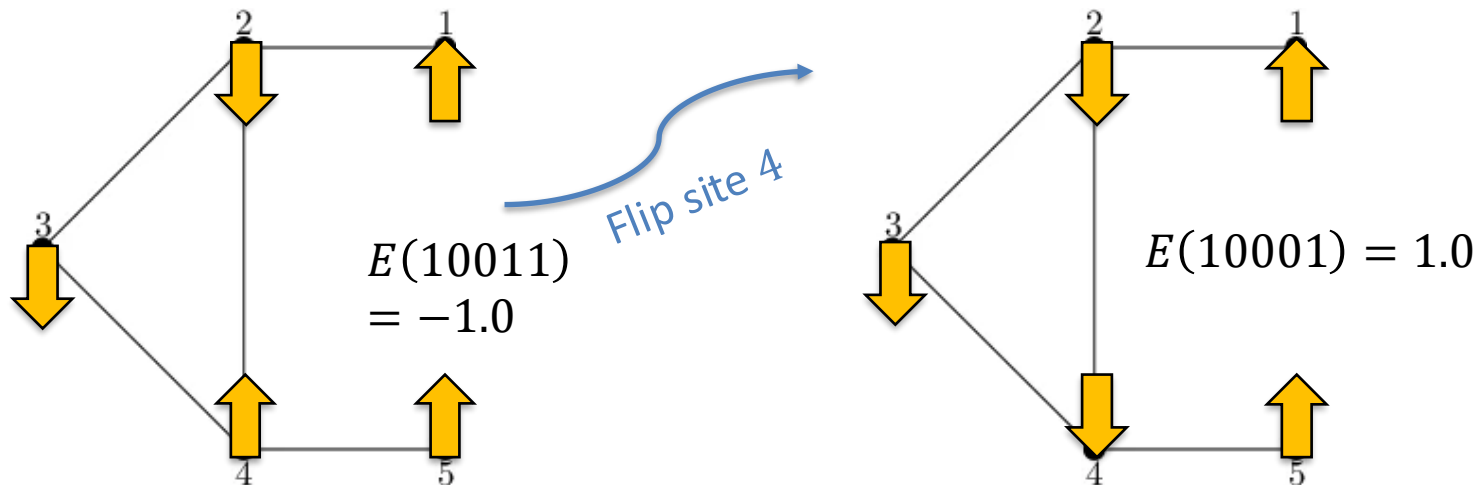
1. Initialize configuration,  $\alpha$
2. Loop over Monte Carlo steps
  1. Loop over sites,  $i$ 
    1. Propose new configuration,  $\beta$ , by flipping site,  $i$ .
    2. Compute flipping probability,  $W(\alpha \rightarrow \beta)$ .
    3. If  $W(\alpha \rightarrow \beta)$  is greater than a randomly chosen number  $r$  between 0 and 1,  
Accept  $\beta$  ( $\alpha \rightarrow \beta$ ),  
else:  
Reject  $\beta$
  2. Update average values with updated  $\alpha$

# Example trajectory

- Consider the graph below, with the configuration 11011
- Assume  $H = \sum_{(i,j) \in E} J_{ij} s_i s_j$ ; and  $J_{ii} = 1.0$  for all  $i, j$



- We accept the configuration 10011



Recall the Metropolis rules

1. If  $E(\beta) - E(\alpha) \leq 0$ ,  
 $W(\alpha \rightarrow \beta) = 1.0$
2. Else, if  
 $E(\beta) - E(\alpha) > 0$ ,  
 $W(\alpha \rightarrow \beta)$   
 $= e^{-\beta(E(\beta) - E(\alpha))}$

$$W(11011 \rightarrow 10011) = ?$$

$$\text{Solution: } W(11011 \rightarrow 10011) = 1.0 \text{ since } E(\beta) - E(\alpha) \leq 0$$

We accept this move if  $e^{-\beta(E(\beta) - E(\alpha))} > r$  for some randomly generated  $r$

$$W(10011 \rightarrow 10001) = ?$$

$$\begin{aligned} \text{Solution: } W(10011 \rightarrow 10001) &= e^{-\beta(E(10001) - E(10011))} \text{ since } E(\beta) - E(\alpha) \leq 0 \\ &= e^{-\beta(1.0 - (-1.0))} = e^{-\beta(2.0)} \cong 0.135 \text{ if } \beta = 1.0 \end{aligned}$$

# Tasks for today

- Pull the assignment `monte\_carlo.ipynb` from the QuantumSoftware GitHub repository
- Activate the virtual environment `molssi\_best\_practices` from earlier
- You will need the package montecarlo from the previous assignment notebook
- Write code to run Monte Carlo simulations of the Ising model
- Calculate the expectation values and record them
- Recall that the expectation value of an observable  $A$  is calculated using

$$\langle A \rangle = \frac{1}{M} \sum_{\alpha}^{\in \text{Trajectory}} A(\alpha)$$